CS155 Miniproject 1: Tuned Ensemble Solution

Predicting Spotify song popularity (High vs Low) from audio features · code: `cs155-miniproject1.py` (285 lines)

1. Summary

The solution keeps the original data cleaning and replaces the single XGBoost model with a **weighted ensemble of six tuned models**: XGBoost, LightGBM, CatBoost, ExtraTrees, RandomForest and logistic regression. Each model's hyperparameters are searched with Optuna, every model is evaluated on cross-validation folds that the search never used, and the models' predictions are combined with weights that are themselves validated on separate folds.

Item	Result
Best cross-validated AUC	**0.8322** (weighted rank blend of 6 models)
Best single model	XGBoost, 0.8310
Blend weights	LightGBM 0.30, RandomForest 0.26, CatBoost 0.18, XGBoost 0.15, ExtraTrees 0.10, LogReg 0.01
Output file	`predictions.csv`: 863 rows, columns `ID, Popularity_Type` (score for High popularity, 0 to 1)

Pipeline at a glance

Step	What happens	Code lines
1-5 Clean	Drop unlabeled rows and URL columns, turn release date into numbers, encode target (High = 1), select features	45-86
6 Define models	Constructor and hyperparameter search space for each of the 6 models	89-176
7 Tune	Optuna search, scored by 5-fold CV grouped by song (seed 42); saved to best_params.json	179-199
8 Evaluate	Refit tuned models on 15 fresh folds (3 repeats x 5, new seeds); collect out-of-fold and test predictions	202-221
9 Ensemble	Convert to ranks, compare 3 blending schemes on held-out folds, pick the best	224-277
10 Predict	Apply chosen weights to test predictions and write predictions.csv	279-285

2. Data preparation (steps 1-5)

The training set has 3,865 rows (one with every field empty) and the test set 863 rows. Classes are imbalanced: 35.2% of songs are High popularity. Steps 2-5 are your original cleaning, kept as written; step 1 and the target encoding were changed.

- **Step 1 (lines 45-49), rows and song id.** Rows with no target are dropped. Before the `uri` column is deleted, it is converted into a numeric song id (`groups`) that the cross-validation uses later (see Section 3).
- **Step 2 (lines 51-54), useless columns.** `track_href, uri, analysis_url, type` are identifiers or constants and carry no audio information.
- **Step 3 (lines 56-69), release date.** Parsed from mixed formats into `release_year, release_month, days_since_release`, and `is_jan1` (a January 1 date usually means only the

year was known).

- **Step 4 (lines 71-73), target.** `y = 1` for High, 0 for Low, matching the assignment. The earlier `LabelEncoder` sorted alphabetically and made High = 0, so the old predictions were the probability of *Low*.
- **Step 5 (lines 75-86), features.** The 17 numeric columns present in both files. `ID` is excluded. The boosting models handle missing values natively; for the sklearn models (ExtraTrees, RandomForest, LogReg) a median-filled copy is made (lines 84-85).

3. Why cross-validation is grouped by song

221 training rows are repeats of a song already in the training set (same `uri`), and 38 of the duplicated songs even carry conflicting labels. None of these songs appear in the test set. With ordinary random folds, one copy of a song can be in the training fold while another copy is in the validation fold, so the model is rewarded for memorising songs it will never see at test time.

This was visible in a first tuning run with ordinary folds: XGBoost scored 0.8388 and the search drifted to the deepest, least-regularised trees allowed. Switching to `StratifiedGroupKFold` with the song id as the group keeps all copies of a song on the same side of every split. The same XGBoost search then scored 0.8282, so about 0.01 of the earlier score was leakage. Grouped folds are used everywhere a split is made: tuning (line 186), evaluation (lines 205-207) and blend validation (line 256).

```
groups = pd.factorize(data_tr['uri'])[0]                      # line 49
tune_cv = StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=42)   # line 186
for tr_idx, va_idx in cv.split(Xn, y, groups):                # line 172
```

4. Models and hyperparameter tuning (steps 6-7)

Why six models. An ensemble only helps if its members make different mistakes. The three gradient-boosting libraries grow trees differently (XGBoost depth-wise, LightGBM leaf-wise, CatBoost symmetric trees with ordered boosting). RandomForest and ExtraTrees average many independent deep trees instead of fitting residuals. Logistic regression on quantile-normalised features is a linear baseline with a very different error pattern.

How tuning works. `make_model` (lines 90-107) builds a model from a parameter dictionary; `suggest` (lines 110-155) defines each search space; `cv_auc` (lines 169-176) returns the 5-fold grouped out-of-fold AUC for one parameter setting. The loop in lines 187-199 runs an Optuna TPE study per model, maximising that AUC, and writes the winners to `best_params.json` after each model so a crash loses nothing. On later runs the file is loaded instead of re-tuning unless `--tune` is passed (lines 180-184). Trial counts are set in line 161: 60 for XGBoost and LightGBM, 30 for CatBoost, ExtraTrees and RandomForest, 20 for LogReg.

Tuned hyperparameters (from best_params.json)

Model	Tuning AUC	Key settings chosen
XGBoost	0.8282	2948 trees, learning rate 0.0026, max depth 9, subsample 0.60, colsample 0.75, almost no L1/L2
LightGBM	0.8275	1478 trees, learning rate 0.0042, 32 leaves, max depth 10, min child samples 16, subsample 0.68
CatBoost	0.8285	695 iterations, learning rate 0.026, depth 6, l2_leaf_reg 5.0, random_strength 2.15
RandomForest	0.8265	500 trees, max depth 16, min samples leaf 2, max_features 0.30, entropy
ExtraTrees	0.8197	900 trees, max depth 30, min samples leaf 3, max_features 0.68, entropy
LogReg	0.7680	C = 7.1 on quantile-normal features

All the tree models prefer many trees, a small learning rate and fairly deep trees. Because this holds even with grouped folds, the data seem to contain real interactions between features. The linear model trails by about 0.06, which supports the same conclusion.

5. Out-of-sample evaluation (step 8)

After tuning, each model is refit with its chosen settings on 15 new splits: 5 grouped folds, repeated 3 times with seeds 2026-2028 (lines 205-207). These seeds differ from the tuning seed, so the split boundaries are new. For every fold the model predicts the held-out songs and the test set (lines 210-221):

- **Out-of-fold predictions** (`oof`): each training row receives one prediction per repeat from a model that never trained on it; the three are averaged.
- **Test predictions** (`test_pred`): the average of all 15 fold models. No final refit on the full data is needed, and the averaging adds a bagging effect.

Model	Out-of-fold AUC	Per-fold mean +/- std
XGBoost	0.8310	0.8277 +/- 0.0114
LightGBM	0.8306	0.8276 +/- 0.0117
RandomForest	0.8305	0.8278 +/- 0.0129
CatBoost	0.8282	0.8242 +/- 0.0113
ExtraTrees	0.8223	0.8199 +/- 0.0117
LogReg	0.7673	0.7671 +/- 0.0186

The out-of-fold AUC pools all rows after averaging 3 predictions per row, which is why it sits a little above the per-fold mean. The fold-to-fold standard deviation of about 0.011 is larger than the gaps between the top four models, so they are statistically close.

6. Ensembling (step 9)

Rank transform (lines 225-228). Each model's predictions are replaced by their rank divided by the number of rows. Different models produce differently scaled probabilities (a random forest's are compressed toward the middle, for example), and AUC depends only on ordering, so blending ranks puts all models on the same footing.

Weight fitting (lines 231-246). `fit_weights` finds non-negative weights summing to 1 that maximise the AUC of the blended ranks. AUC is not differentiable, so it uses the derivative-free Nelder-Mead method, started from equal weights and from 6 starts that each lean toward one model, and keeps the

best.

Honest comparison (lines 249-259). Three schemes are scored. The weighted blend is scored "nested": weights are learned on 4/5 of the songs' out-of-fold predictions and scored on the remaining 1/5, five times, so the weights never see the rows they are judged on. The best scheme is chosen in lines 270-276; the final weights are refit on all rows (line 261).

Ensemble	AUC
Equal-weight average of all 6 models	0.8304
Equal-weight average of XGBoost, LightGBM, CatBoost	0.8314
Weighted rank blend (nested validation), used for submission	**0.8322**

The equal average of all six is dragged down by logistic regression, which the weighted blend gives almost zero weight. RandomForest gets a large weight (0.26) despite not being the top single model: its errors differ from the boosting models', which is what an ensemble rewards.

7. Test predictions (step 10)

Lines 280-284 rank-transform each model's averaged test predictions, combine them with the chosen weights, and write `predictions.csv` with columns `ID` and `Popularity_Type`. The values are blended rank scores between 0 and 1, not calibrated probabilities; this does not affect AUC, which uses only the ordering. Check the column names against Kaggle's sample submission before uploading.

8. How "out of sample" is the 0.832?

Every prediction behind the score was made by a model that did not train on that row, the evaluation folds are different from the tuning folds, and duplicate songs never straddle a split. However, the same 3,864 rows were used to *choose* the hyperparameters, so the number is somewhat optimistic. The bias is expected to be small (perhaps 0.002-0.005) because the search was modest and fold-to-fold noise is about 0.011, but it has not been measured. A clean estimate would require a locked holdout of about 20% of songs that is never used in tuning, or nested cross-validation with tuning repeated inside each outer fold. The public Kaggle leaderboard gives a third, independent check.

9. Code map

Line numbers refer to the current `cs155-miniproject1.py`.

Lines	Code	Role in the solution
1-30	Docstring, imports	Usage notes; libraries for the 6 models, Optuna, grouped CV, AUC
32-38	argparse, PARAMS_FILE	`--tune` re-runs the search, `--trials` sets its size; where tuned settings are cached
40-42	read_csv	Load train.csv and test.csv
45-49	Step 1	Drop rows with no target; build song-id `groups` for grouped CV (Section 3)
51-54	Step 2	Drop identifier and constant columns
56-69	Step 3: parse_date	Release date to year, month, days since release, Jan-1 flag
71-73	Step 4	Target y = 1 for High popularity
75-86	Step 5	Choose 17 numeric features; median-filled copy for sklearn models
90-107	make_model	Build any of the 6 models from a parameter dict and seed
110-155	suggest	Optuna search space for each model
158-163	MODELS, NEEDS_FILLED, TRIALS	Model list; which models need filled data; trials per model
165-166	data_for	Return raw or median-filled features for a model
169-176	cv_auc	Grouped 5-fold out-of-fold AUC for one setting (the tuning objective)
180-184	load best_params.json	Skip tuning when cached settings exist and --tune is not set
186-199	Step 7: tuning loop	Optuna TPE study per model on seed-42 grouped folds; save winners
205-207	eval_splits	15 fresh grouped splits (3 repeats x 5 folds, seeds 2026-2028)
210-221	Step 8: evaluation loop	Fit each tuned model per fold; collect out-of-fold and averaged test predictions; print AUCs
225-228	rank01, O	Convert predictions to ranks so models are on the same scale
231-246	fit_weights	Nelder-Mead with 7 starts: non-negative weights maximising blend AUC
249-259	results, nested	Score equal-all, equal-GBM and weighted blends; weighted is scored on held-out folds
261-268	w_final, report	Refit weights on all rows; print summary table
270-277	choice	Select the blending scheme with the best validated AUC
280-285	Step 10	Blend test ranks with chosen weights; write predictions.csv

How to run

```
python cs155-miniproject1.py                  # ~3 min, reuses best_params.json
python cs155-miniproject1.py --tune           # ~40 min, full re-tune
python cs155-miniproject1.py --tune --trials 80
```

Files produced

- `predictions.csv`: the submission (ID, score for High popularity).
- `best_params.json`: tuned hyperparameters for all 6 models.
- `tune_log.txt`: full console output of the tuning run, useful for the report's model-selection section.